# Weighted graphs. Optimization on graphs (1).

Przemysław Gordinowicz

Institute of Mathematics,
Łódź University of Technology


Graphs & Social Networks
Lecture 6

# Outline

1. Minimum spanning trees
2. Shortest paths
3. Other problems

# Outline

1 Minimum spanning trees

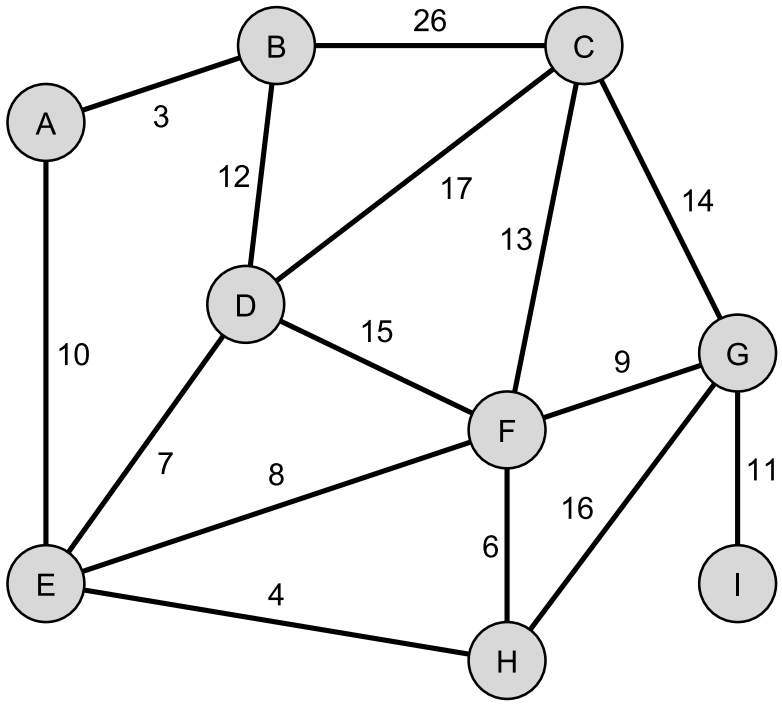
2 Shortest paths

3 Other problems

# The problem of railway builders

**Problem (cf. D. Harel „Algorithmics. The spirit of computing ”)**

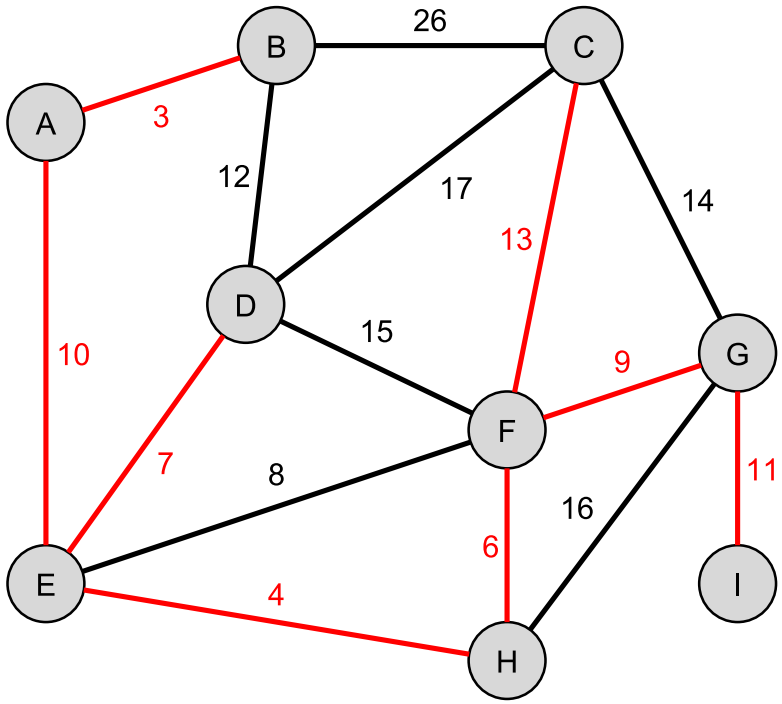
*Knowing the cost of building a direct rail connection between each pair of cities (for which such a connection is possible), design the **cheapest** rail network connecting **all** cities.*



# The problem of railway builders

**Problem (cf. D. Harel „Algorithmics. The spirit of computing ”)**

*Knowing the cost of building a direct rail connection between each pair of cities (for which such a connection is possible), design the **cheapest** rail network connecting **all** cities.*



# Weighted graphs

## Definition

A **weighted graph** is a pair $(G, w)$, where $G$ is a graph (simple or multigraph, undirected or directed), and $w\colon E(G) \to \mathbb{R}$ (for directed ones $w\colon A(G) \to \mathbb{R}$) is a **function of weights**.

## Remark

In many problems (eg. the one of railway builders) weights are nonnegative. Sometimes it is useful to extend the weight set by $\{-\infty, \infty\}$. For simple graphs, one can redefine the function of weights to pair of vertices, in a natural way

$$w\colon V(G) \times V(G) \to \mathbb{R} \cup \{-\infty, \infty\}.$$

# Minimum spanning tree

The problem of railway builders in the language of graph theory means: given weighted graph $(G, w)$ with $G$ undirected and connected, find a spanning tree $T = (V(G), E)$, that minimizes

$$\sum_{e \in E} w(e).$$

**Problem**

*The number of spanning tree may be even $n^{n-2}$, for $n = |V(G)|$. Eg. for $n = 20 : 20^{18} \approx 2,6 * 10^{23}$.*

# Being greedy (sometimes) pays off!

## Algorithm of Prim[1957] or rather of Jarnik[1930]

**Function** Jarnik_Prim($G$, $w$, $v$)
  $V_T := \{v\}$;
  $E_T := \emptyset$;
  **while** $\exists uw \in E(G)\ u \in V_T \wedge w \in V(G) \setminus V_T$ **do**
    $uw :=$ edge with the least weight, $u \in V_T$, $w \notin V_T$;
    $(V_T, E_T) := (V_T, E_T) + (w, uw)$;
  Jarnik_Prim $:= (V_T, E_T)$;

Correctnes proof (sketch):
As a result we get a spanning tree — that was already there.
Why does being greedy pay off here?

# Being greedy (sometimes) pays off!

## Algorithm of Prim[1957] or rather of Jarnik[1930]

**Function** Jarnik_Prim($G$, $w$, $v$)
    $V_T := \{v\}$;
    $E_T := \emptyset$;
    **while** $V_T \neq V(G)$ **do**
        $uw :=$ edge with the least weight, $u \in V_T$, $w \notin V_T$;
        $(V_T, E_T) := (V_T, E_T) + (w, uw)$;
    Jarnik_Prim $:= (V_T, E_T)$;

Correctnes proof (sketch):
As a result we get a spanning tree — that was already there.
Why does being greedy pay off here?

# Being greedy (sometimes) pays off!

## Algorithm of Prim[1957] or rather of Jarnik[1930]

**Function** Jarnik_Prim($G$, $w$, $v$)
  $V_T := \{v\}$;
  $E_T := \emptyset$;
  **while** $V_T \neq V(G)$ **do**
    $uw :=$ edge with the least weight, $u \in V_T$, $w \notin V_T$;
    $(V_T, E_T) := (V_T, E_T) + (w, uw)$;
  Jarnik_Prim $:= (V_T, E_T)$;

Correctnes proof (sketch):
As a result we get a spanning tree — that was already there.
Why does being greedy pay off here?

# Being greedy (sometimes) pays off!

## Algorithm of Prim[1957] or rather of Jarnik[1930]

**Function** Jarnik_Prim($G$, $w$, $v$)
    $V_T := \{v\}$;
    $E_T := \emptyset$;
    **while** $V_T \neq V(G)$ **do**
        $uw :=$ edge with the least weight, $u \in V_T$, $w \notin V_T$;
        $(V_T, E_T) := (V_T, E_T) + (w, uw)$;
    Jarnik_Prim $:= (V_T, E_T)$;

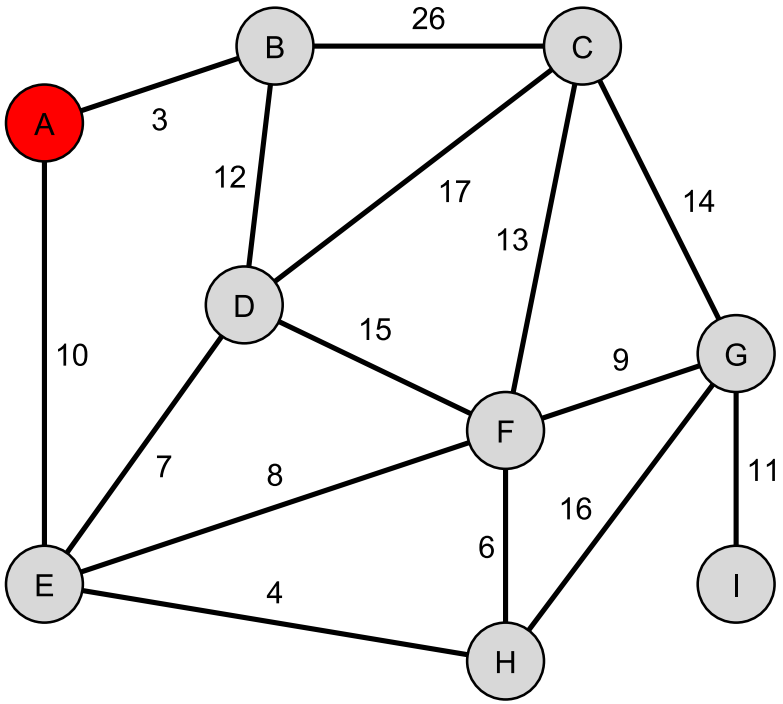
In the implementation, priority queues (e.g. binary heaps or Fibonacci heaps) are used to select the edges with the least weight.

The computational complexity (using adjacency lists) is $O(|E| \lg |V|)$ (binary heaps) or $O(|E| + |V| \lg |V|)$ (Fibonacci heaps).

Using adjacency matrices, the complexity is easily $O(|V^2|)$.

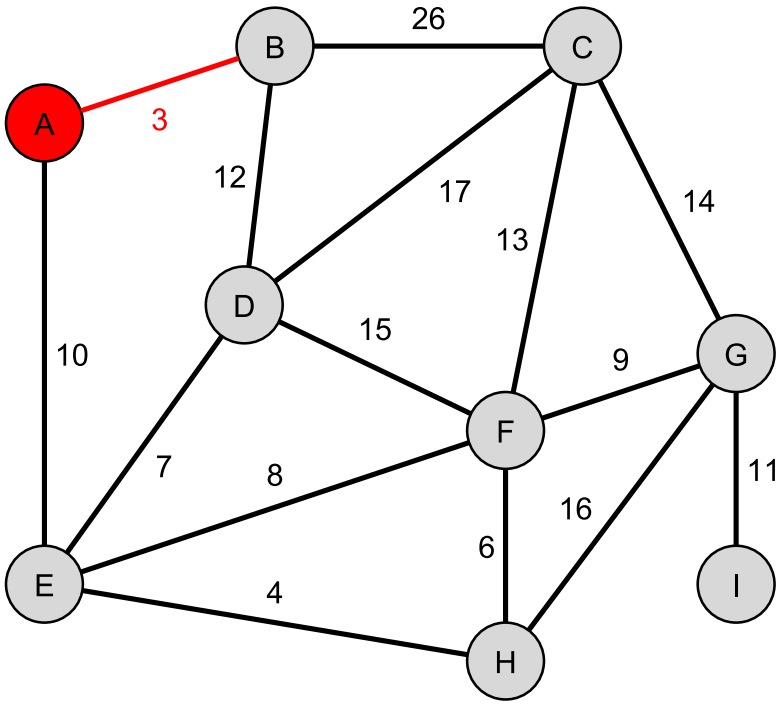
# Being greedy (sometimes) pays off!

**Function** Jarnik_Prim($G$, $w$, $v$)
  $V_T := \{v\}$;
  $E_T := \emptyset$;
  **while** $V_T \neq V(G)$ **do**
    $uw :=$ edge with the least weight, $u \in V_T$, $w \notin V_T$;
    $(V_T, E_T) := (V_T, E_T) + (w, uw)$;
  Jarnik_Prim $:= (V_T, E_T)$;



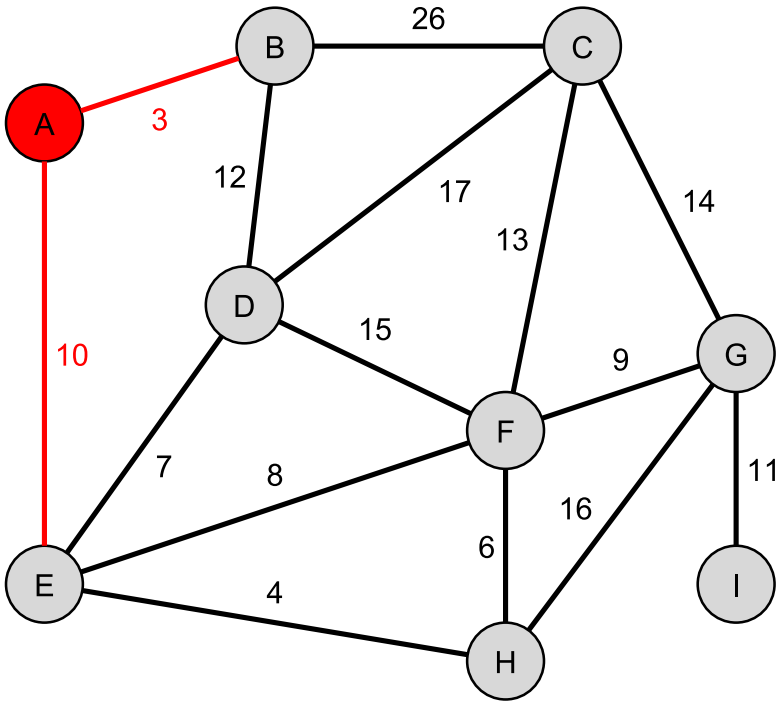
# Being greedy (sometimes) pays off!

**Function** Jarnik_Prim($G$, $w$, $v$)
  $V_T := \{v\}$;
  $E_T := \emptyset$;
  **while** $V_T \neq V(G)$ **do**
    $uw :=$ edge with the least weight, $u \in V_T$, $w \notin V_T$;
    $(V_T, E_T) := (V_T, E_T) + (w, uw)$;
  Jarnik_Prim $:= (V_T, E_T)$;



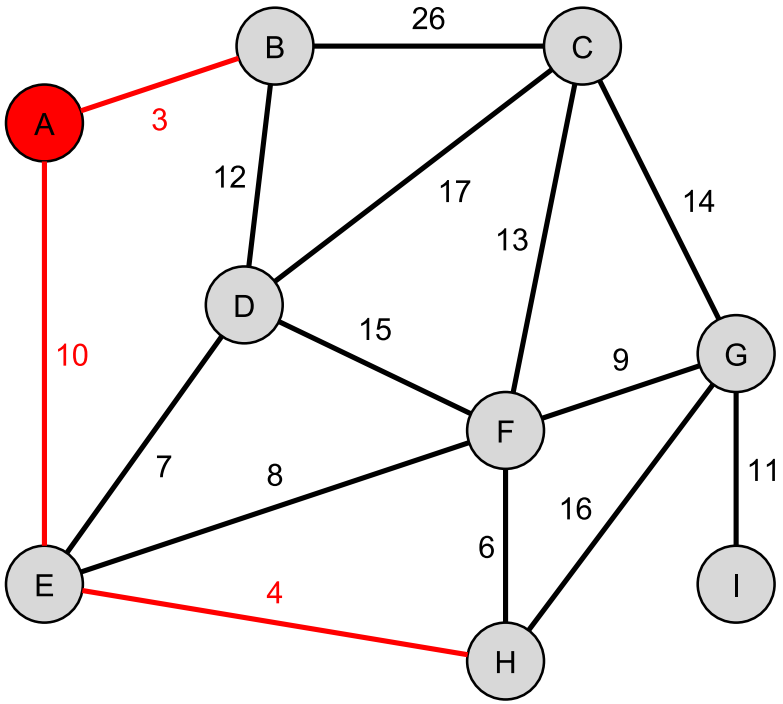
# Being greedy (sometimes) pays off!

**Function** Jarnik_Prim($G$, $w$, $v$)
  $V_T := \{v\}$;
  $E_T := \emptyset$;
  **while** $V_T \neq V(G)$ **do**
    $uw :=$ edge with the least weight, $u \in V_T$, $w \notin V_T$;
    $(V_T, E_T) := (V_T, E_T) + (w, uw)$;
  Jarnik_Prim $:= (V_T, E_T)$;



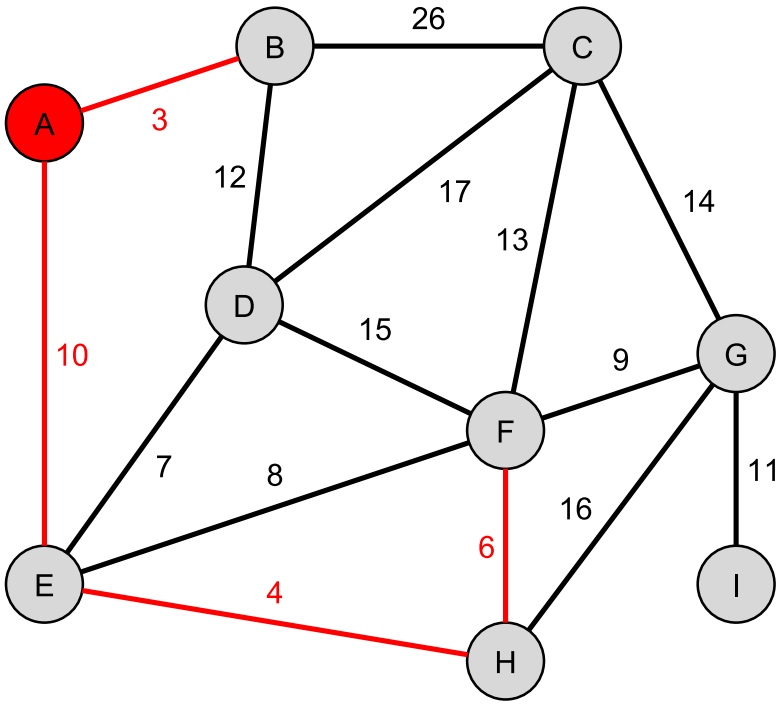
# Being greedy (sometimes) pays off!

**Function** Jarnik_Prim($G$, $w$, $v$)
  $V_T := \{v\}$;
  $E_T := \emptyset$;
  **while** $V_T \neq V(G)$ **do**
    $uw :=$ edge with the least weight, $u \in V_T$, $w \notin V_T$;
    $(V_T, E_T) := (V_T, E_T) + (w, uw)$;
  Jarnik_Prim $:= (V_T, E_T)$;



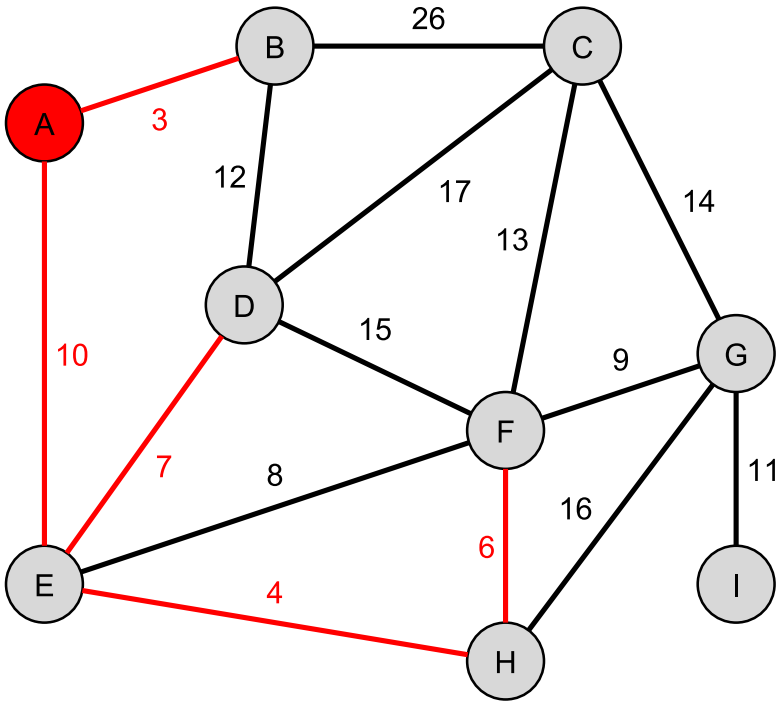
# Being greedy (sometimes) pays off!

**Function** Jarnik_Prim($G$, $w$, $v$)
  $V_T := \{v\}$;
  $E_T := \emptyset$;
  **while** $V_T \neq V(G)$ **do**
    $uw :=$ edge with the least weight, $u \in V_T$, $w \notin V_T$;
    $(V_T, E_T) := (V_T, E_T) + (w, uw)$;
  Jarnik_Prim $:= (V_T, E_T)$;



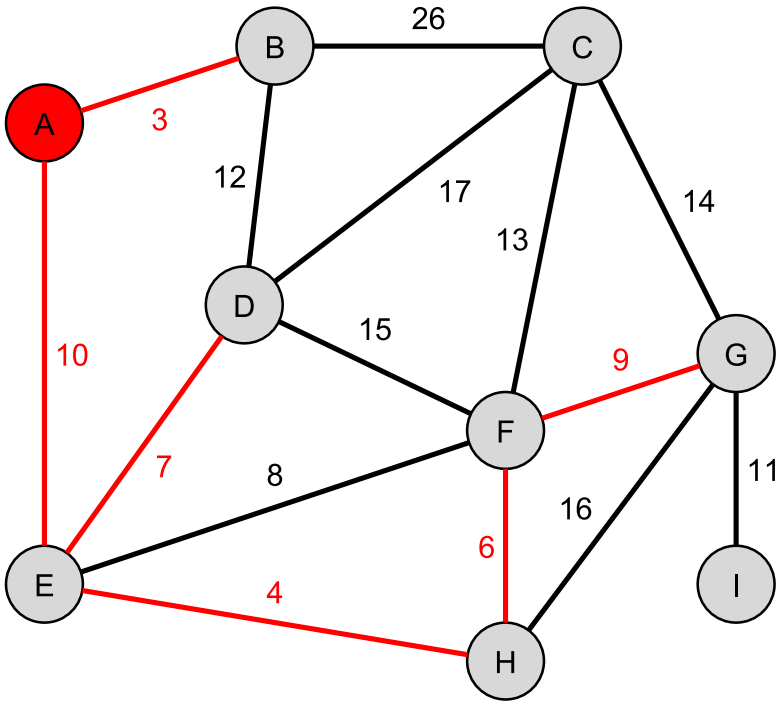
# Being greedy (sometimes) pays off!

**Function** Jarnik_Prim($G$, $w$, $v$)
  $V_T := \{v\}$;
  $E_T := \emptyset$;
  **while** $V_T \neq V(G)$ **do**
    $uw :=$ edge with the least weight, $u \in V_T$, $w \notin V_T$;
    $(V_T, E_T) := (V_T, E_T) + (w, uw)$;
  Jarnik_Prim $:= (V_T, E_T)$;



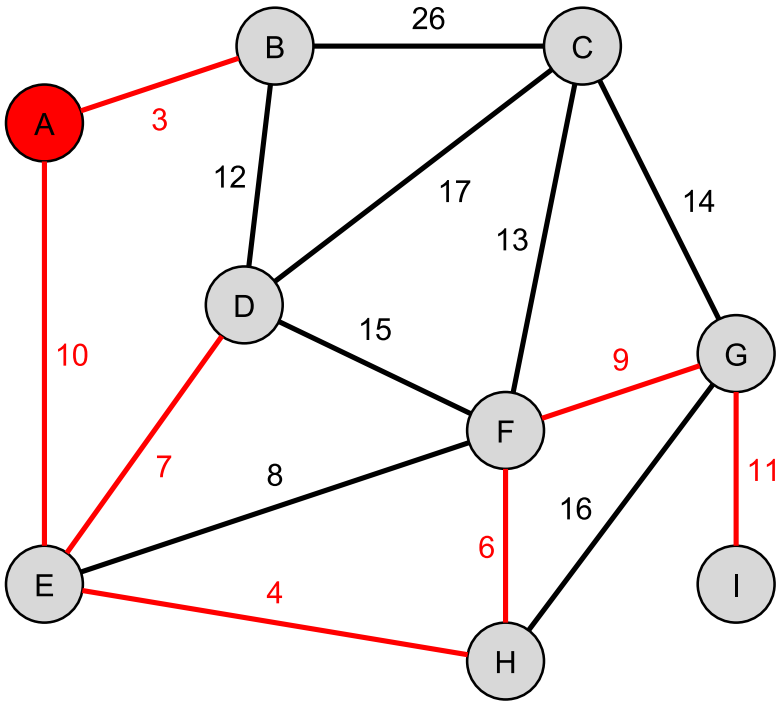
# Being greedy (sometimes) pays off!

**Function** Jarnik_Prim($G$, $w$, $v$)
  $V_T := \{v\}$;
  $E_T := \emptyset$;
  **while** $V_T \neq V(G)$ **do**
    $uw :=$ edge with the least weight, $u \in V_T$, $w \notin V_T$;
    $(V_T, E_T) := (V_T, E_T) + (w, uw)$;
  Jarnik_Prim $:= (V_T, E_T)$;



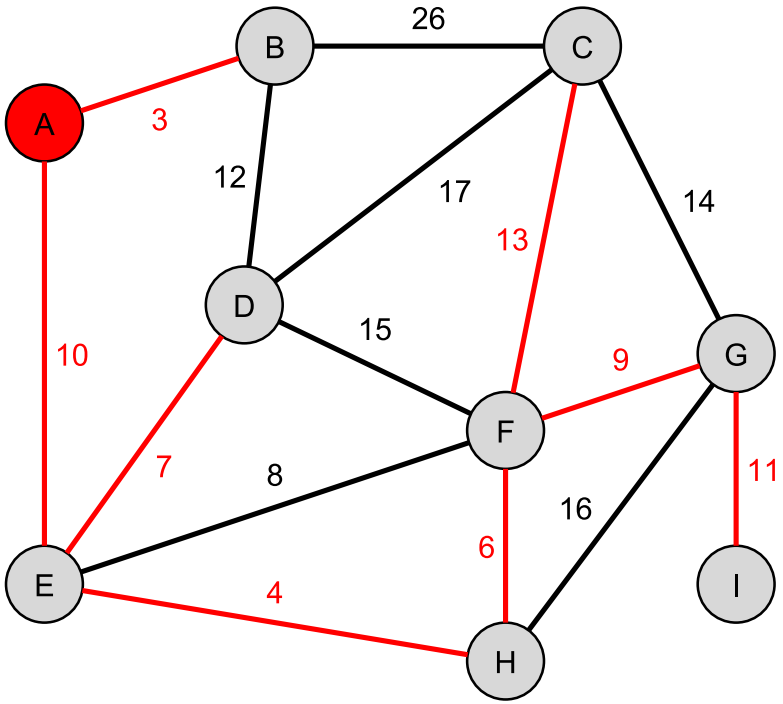
# Being greedy (sometimes) pays off!

**Function** Jarnik_Prim($G$, $w$, $v$)
  $V_T := \{v\}$;
  $E_T := \emptyset$;
  **while** $V_T \neq V(G)$ **do**
    $uw :=$ edge with the least weight, $u \in V_T$, $w \notin V_T$;
    $(V_T, E_T) := (V_T, E_T) + (w, uw)$;
  Jarnik_Prim $:= (V_T, E_T)$;



# Being greedy (sometimes) pays off!

**Function** Jarnik_Prim($G$, $w$, $v$)
  $V_T := \{v\}$;
  $E_T := \emptyset$;
  **while** $V_T \neq V(G)$ **do**
    $uw :=$ edge with the least weight, $u \in V_T$, $w \notin V_T$;
    $(V_T, E_T) := (V_T, E_T) + (w, uw)$;
  Jarnik_Prim $:= (V_T, E_T)$;



# Being greedy (sometimes) pays off!

## Kruskal's algorithm[1956]

**Function** Kruskal($G$, $w$,)
  $E_T := \emptyset$;
  sort $E(G)$ non-increasing according to weights;
  **for each** $e \in E(G)$ (in given order) **do**
    **if** graph $(V(G), E_T \cup \{e\})$ is acyclic
      **then** $E_T := E_T \cup \{e\}$;
  Kruskal $:= (V(G), E_T)$;

Kruskal's algorithm for disconnected graphs returns a forest of minimum spanning trees of every connected component. The difficulty in efficient implementation is the checking for acyclicity — this is done by numbering the components (and renumbering them when merging). The computational complexity is $O(|E| \lg |E|)$ (implementation on adjacency lists).

# Being greedy (sometimes) pays off!

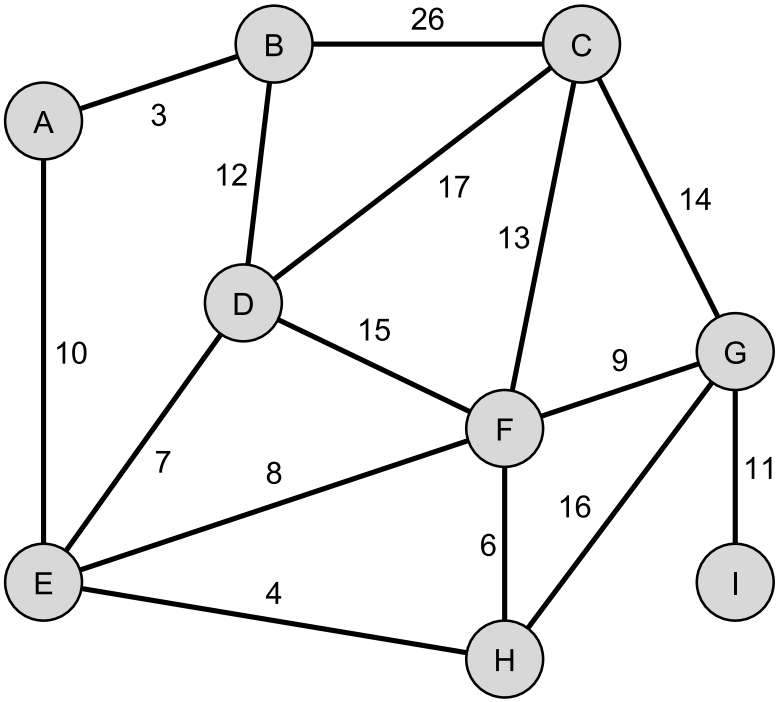
## Kruskal's algorithm[1956]

**Function** Kruskal($G$, $w$,)
  $E_T := \emptyset$;
  sort $E(G)$ non-increasing according to weights;
  **for each** $e \in E(G)$ (in given order) **do**
    **if** graph $(V(G), E_T \cup \{e\})$ is acyclic
      **then** $E_T := E_T \cup \{e\}$;
  Kruskal $:= (V(G), E_T)$;

Kruskal's algorithm for disconnected graphs returns a forest of minimum spanning trees of every connected component. The difficulty in efficient implementation is the checking for acyclicity — this is done by numbering the components (and renumbering them when merging). The computational complexity is $O(|E| \lg |E|)$ (implementation on adjacency lists).

# Being greedy (sometimes) pays off!

**Function** Kruskal($G$, $w$,)
  $E_T := \emptyset$;
  sort $E(G)$ non-increasing according to weights;
  **for each** $e \in E(G)$ (in given order) **do**
    **if** graph $(V(G), E_T \cup \{e\})$ is acyclic
      **then** $E_T := E_T \cup \{e\}$;
  Kruskal $:= (V(G), E_T)$;




A
B
C
D
E
F
G
H
I

# Outline

1 Minimum spanning trees

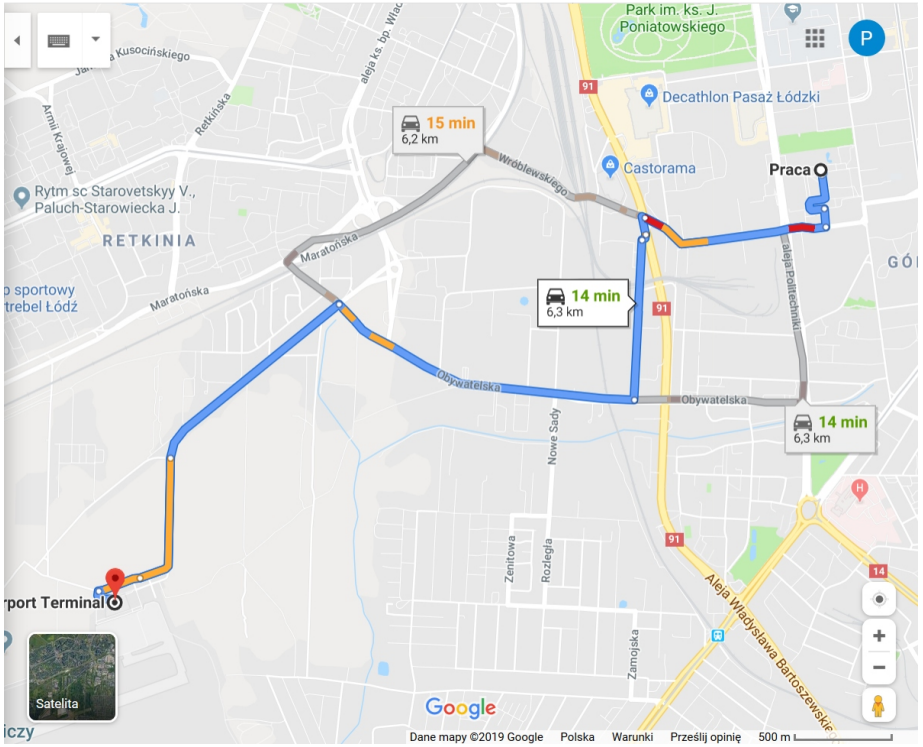
2 Shortest paths

3 Other problems

# Shortest paths (weary hikers)

Based on distances between directly connected points on the map, compute:

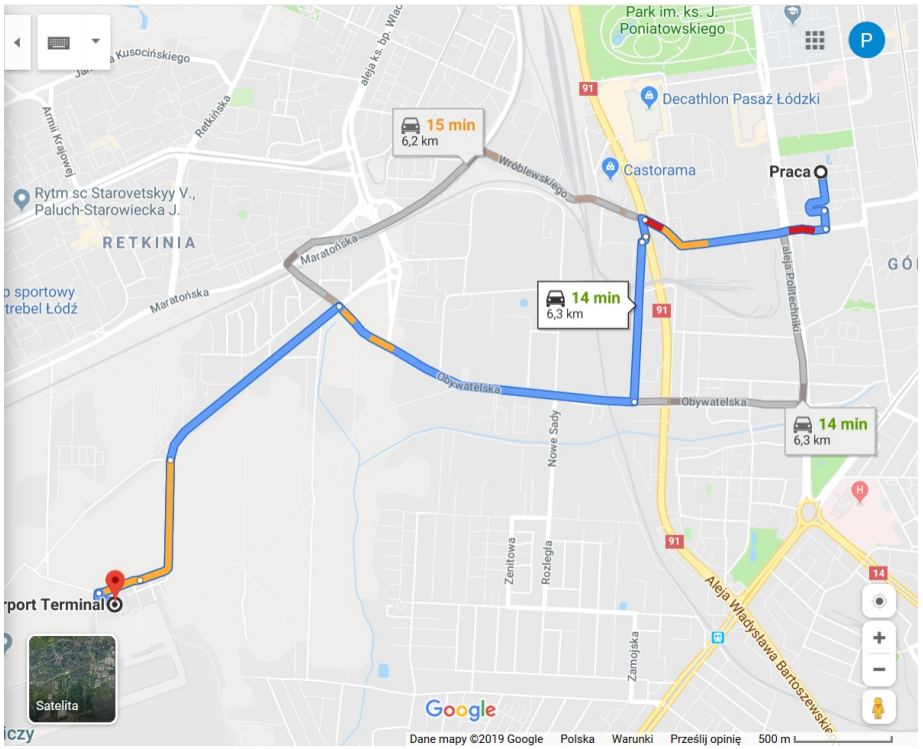
- given points *a* and *b*, the shortest path from the point *a* to the point *b*
- shortest paths from the point *a* to any other point on the map (single-source shortest paths)
- the distance matrix for all points on the map



# Shortest paths (weary hikers)

Based on distances between directly connected points on the map, compute:

- given points *a* and *b*, the shortest path from the point *a* to the point *b*
- **shortest paths from the point *a* to any other point on the map (single-source shortest paths)**
- the distance matrix for all points on the map



# Triangle inequality

Let $\mathrm{dist}\colon V(G) \times V(G) \to \mathbb{R} \cup \{\infty\}$ denote the distance (the length of the shortest path) between vertices in a weighted graph.

**Lemma**

*Let $(G, w)$ be a weighed, directed graph with non-negative weights, let $s, u, v \in V(G)$. Then*

$$\mathrm{dist}(s, v) \leq \mathrm{dist}(s, u) + \mathrm{dist}(u, v).$$

*Moreover, for $uv \in A(G)$ there is $\mathrm{dist}(u, v) \leq w(u, v)$.*

For undirected graphs — analogously.

**Corollary**

*For weighted undirected graphs with positive weights function $\mathrm{dist}$ is a metric on the vertex set.*

# Triangle inequality

Let $\mathrm{dist}\colon V(G) \times V(G) \to \mathbb{R} \cup \{\infty\}$ denote the distance (the length of the shortest path) between vertices in a weighted graph.

## Lemma

*Let $(G, w)$ be a weighed, directed graph with non-negative weights, let $s, u, v \in V(G)$. Then*

$$\mathrm{dist}(s, v) \leq \mathrm{dist}(s, u) + \mathrm{dist}(u, v).$$

*Moreover, for $uv \in A(G)$ there is $\mathrm{dist}(u, v) \leq w(u, v)$.*

For undirected graphs — analogously.

## Corollary

*For weighted undirected graphs with positive weights function $\mathrm{dist}$ is a metric on the vertex set.*

# Shortest paths algorithm — initialisation

We describe an algorithm for computing shortest paths from a single source $s$ in a directed graph $(G, w)$ with non-negative weights. Let us denote $\mathrm{dist}(v) := \mathrm{dist}(s, v)$, for $v \in V(G)$. The distance computing algorithm will proceed in several steps.

Initialization of the distance table and the predecessor table.

**Procedure** Init$(G, s)$;
    **for each** $v \in V(G)$ **do**
        $\mathrm{dist}(v) := \infty$;
        $\mathrm{pred}(v) := \emptyset$;
    $\mathrm{dist}(s) := 0$;

# Triangle inequality — an algorithmic version

From Lemma (two slides back) we have:

$$\mathrm{dist}(s, v) \leq \mathrm{dist}(s, u) + w(u, v)$$

for each arc $uv \in A(G)$.

This condition is used to improve the distance computed up to a given step, by doing (so called) arc relaxation:

**Procedure** Relax$(u, v)$;
  **if** $\mathrm{dist}(v) > \mathrm{dist}(u) + w(u, v)$ **then**
    $\mathrm{dist}(v) := \mathrm{dist}(u) + w(u, v)$;
    $\mathrm{pred}(v) := u$;

Intuition: by performing multiple arc relaxations, we will eventually obtain the shortest paths.

# Triangle inequality — an algorithmic version

From Lemma (two slides back) we have:

$$\mathrm{dist}(s, v) \leq \mathrm{dist}(s, u) + w(u, v)$$

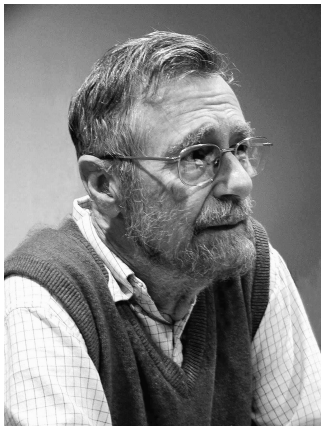
for each arc $uv \in A(G)$.

This condition is used to improve the distance computed up to a given step, by doing (so called) arc relaxation:

**Procedure** Relax$(u, v)$;
  **if** $\mathrm{dist}(v) > \mathrm{dist}(u) + w(u, v)$ **then**
    $\mathrm{dist}(v) := \mathrm{dist}(u) + w(u, v)$;
    $\mathrm{pred}(v) := u$;

Intuition: by performing multiple arc relaxations, we will eventually obtain the shortest paths.

# Dijkstra's algorithm



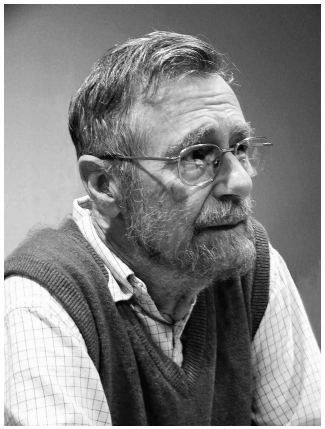
Edsger Dijkstra
(1930–2002)

## Dijkstra's algorithm [1959]

**Procedure** Dijkstra($G, w, s$);
    Init($G, s$);
    $V = \emptyset$;
    **while** $V \neq V(G)$ **do**
        $u := \arg\min\{\mathrm{dist}(v) \colon v \in V(G) \setminus V\}$;
        $V := V \cup \{u\}$;
        **for** $v \in N^+(u) \setminus V$ **do**
            Relax($u, v$);

Depending on the implementation, one can achieve complexity of $O(|V|^2)$ (naive version), $O(|A| \lg |V|)$ (binary heaps) or even $O(|V| \lg |V| + |A|)$ (Fibonacci heaps).

# Dijkstra's algorithm



Edsger Dijkstra
(1930–2002)

## Dijkstra's algorithm [1959]

**Procedure** Dijkstra$(G, w, s)$;
    Init$(G, s)$;
    $V = \emptyset$;
    **while** $V \neq V(G)$ **do**
        $u := \arg\min\{\mathrm{dist}(v) \colon v \in V(G) \setminus V\}$;
        $V := V \cup \{u\}$;
        **for** $v \in N^+(u) \setminus V$ **do**
            Relax$(u, v)$;

Depending on the implementation, one can achieve complexity of $O(|V|^2)$ (naive version), $O(|A| \lg |V|)$ (binary heaps) or even $O(|V| \lg |V| + |A|)$ (Fibonacci heaps).

# Bellman–Ford algorithm

## Bellman–Ford algorithm [1958] for directed graphs

**Procedure** Bellman-Ford($G, w, s$);
    Init($G, s$);
    **for** $i := 1$ **until** $|V(G)| - 1$ **do**
        **for each** $uv \in A(G)$ **do**
            Relax($u, v$);

Bellman–Ford algorithm has computational complexity of $O(|V||A|)$, which is much worse than Dijkstra's algorithm.

# Bellman–Ford algorithm

## Bellman–Ford algorithm [1958] for directed graphs

**Procedure** Bellman-Ford($G, w, s$);
    Init($G, s$);
    **for** $i := 1$ **until** $|V(G)| - 1$ **do**
        **for each** $uv \in A(G)$ **do**
            Relax($u, v$);

Bellman–Ford algorithm has computational complexity of $O(|V||A|)$, which is much worse than Dijkstra's algorithm.

# Bellman–Ford algorithm

## Bellman–Ford algorithm [1958] for directed graphs

**Procedure** Bellman-Ford($G, w, s$);
    Init($G, s$);
    **for** $i := 1$ **until** $|V(G)| - 1$ **do**
        **for each** $uv \in A(G)$ **do**
            Relax($u, v$);

Bellman–Ford algorithm has computational complexity of $O(|V||A|)$, which is much worse than Dijkstra's algorithm, but the algorithm works well even when there are negative weights! Provided, of course, that there are no negative-weight cycles.

# Bellman–Ford algorithm

## Bellman–Ford algorithm [1958] for directed graphs

**Procedure** Bellman-Ford($G, w, s$);
    Init($G, s$);
    **for** $i := 1$ **until** $|V(G)| - 1$ **do**
        **for each** $uv \in A(G)$ **do**
            Relax($u, v$);

**Function** Bellman-Ford_check($G, w, s$);
    Bellman-Ford($G, w, s$);
    **for each** $uv \in A(G)$ **do**
        **if** $\mathrm{dist}(v) > \mathrm{dist}(u) + w(u, v)$ **then**
            Bellman-Ford_check := „negative cycle”;
    Bellman-Ford_check := „OK”;

# Floyd–Warshall algorithm

A very simple algorithm for computing the distance matrix between all pairs of vertices. The input is a matrix $W \in M(n, n)$ of direct distances in the graph.

## Floyd–Warshall algorithm [1962]

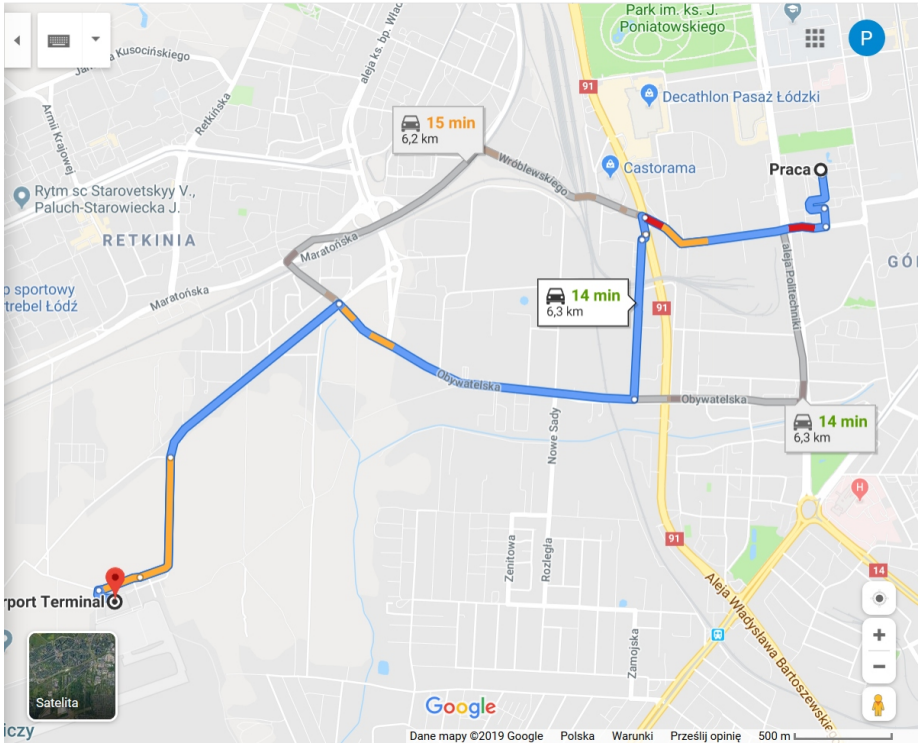
**Function** Floyd-Warshall($W$);
    $D := W$;
    $n := \dim(W)$;
    **for** $k := 1$ **until** $n$ **do**
        **for** $i := 1$ **until** $n$ **do**
            **for** $j := 1$ **until** $n$ **do**
                $d_{ij} := \min(d_{ij}, d_{ik} + d_{kj})$;
    Floyd-Warshall $:= D$;

The algorithm can easily be extended to include information about the penultimate vertex on each path.

# Shortest paths on-line (or how to avoid traffic jams)

Based on distances between directly connected points on the map, compute:

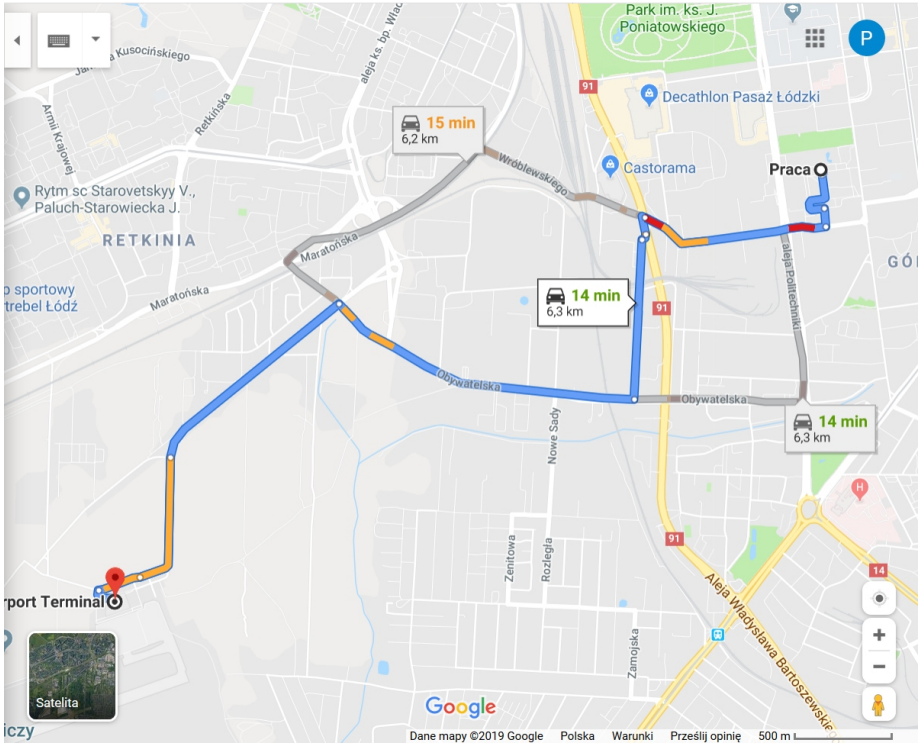
- **given points *a* and *b*, the shortest path from the point *a* to the point *b***
- shortest paths from the point *a* to any other point on the map (single-source shortest paths)
- the distance matrix for all points on the map



# Shortest paths on-line (or how to avoid traffic jams)

Knowing the current travel times between directly connected points on the map, determine the fastest path from the point *a* to the point *b*.

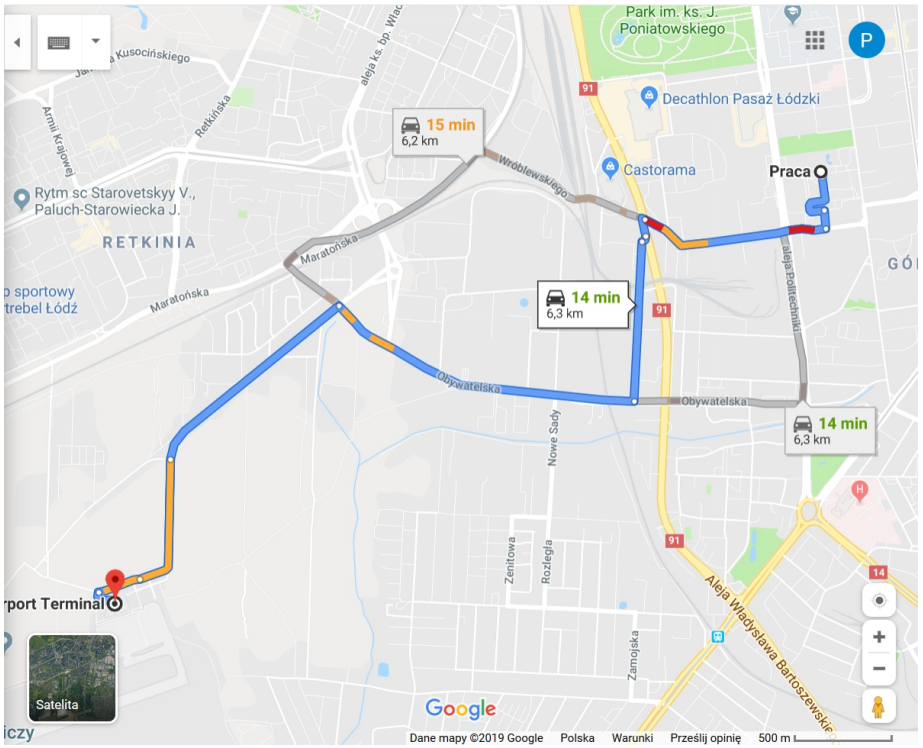
You can use Dijkstra's algorithm (and stop after reaching point *b*).



# Shortest paths on-line (or how to avoid traffic jams)

Knowing the current travel times between directly connected points on the map and the optimal travel times between (*not necessarily!*) all points on the map (*heuristics*), determine the fastest path from the point *a* to the point *b*.

Algorithm A*.



# Outline

1 Minimum spanning trees

2 Shortest paths

3 Other problems

# Longest paths

Does it make sense to determine longest paths?

Yes, in directed acyclic graph it makes sense.

Do we know an algorithm for this?

To be honest, even two ;-)

# Longest paths

Does it make sense to determine longest paths?

Yes, in directed acyclic graph it makes sense.

Do we know an algorithm for this?

To be honest, even two ;-)

# Longest paths

Does it make sense to determine longest paths?

Yes, in directed acyclic graph it makes sense.

Do we know an algorithm for this?

To be honest, even two ;-)

# Longest paths

Does it make sense to determine longest paths?

Yes, in directed acyclic graph it makes sense.

Do we know an algorithm for this?

To be honest, even two ;-)

# The Traveling Salesman and the (Chinese) Postman Problem

A travelling salesman has to visit $n$ cities and return back home. Knowing the distances between the cities, determine the shortest route for him.

A postman has to walk all the streets in his area and return to the post office. Knowing the lengths of the streets, determine the shortest route for the postman.

More — in the next lecture.

# The Traveling Salesman and the (Chinese) Postman Problem

A travelling salesman has to visit $n$ cities and return back home. Knowing the distances between the cities, determine the shortest route for him.

A postman has to walk all the streets in his area and return to the post office. Knowing the lengths of the streets, determine the shortest route for the postman.

More — in the next lecture.

# The Traveling Salesman and the (Chinese) Postman Problem

A travelling salesman has to visit $n$ cities and return back home. Knowing the distances between the cities, determine the shortest route for him.

A postman has to walk all the streets in his area and return to the post office. Knowing the lengths of the streets, determine the shortest route for the postman.

More — in the next lecture.

# Maximum flow problem

Considering a directed graph with distinguished vertices — a *source* and a *sink*, with weights indicating *capacity* of arcs, we determine the maximum flow (how much of substance can be sent from the source to the sink).

More — in three weeks.

# Maximum flow problem

Considering a directed graph with distinguished vertices — a *source* and a *sink*, with weights indicating *capacity* of arcs, we determine the maximum flow (how much of substance can be sent from the source to the sink).

More — in three weeks.